Mandate — one page, for a Rain engineer

Team 10. Hand them this, not the repo. Two minutes to read.

What we built, in one sentence

> **Rain bounds how much an agent spends and where. We check *why* — the agent has to declare the purchase order it negotiated, deterministic code checks that against the real record, and if it doesn't hold, no card is ever issued.**

Not a decline. There is nothing to decline, because the card was never created.

If that sounds like a database constraint: it's a **three-way match**, the same PO / receipt / invoice check every ERP runs — moved from *after the invoice arrives*, where the remedy is a dispute, to *before the instrument exists*.

What actually works

- **11 deterministic checks.** No model, no I/O, no wall clock in the decision path.
- **Rules are versioned data**, not code. Changing one needs a second person to approve it — the author can't approve their own change.
- **Replay.** Edit a rule, re-run all 54 recorded decisions against it, see what flips. Only meaningful because nothing in the check path is a model.
- **Append-only log** storing a *snapshot* of the record read, not a pointer to it.
- **Negotiation** — four sellers, distinct strategies, one counter-offer round. The winner becomes the PO that gets checked.
- **Idempotency under concurrency.** Two identical requests at the same instant produce exactly one card. (This was a real bug we found and fixed — rule 6 only protected the sequential case.)
- **82 tests.**

Cards are currently **simulated and labelled as such** in the UI. See the blockers below.

🔴 Three questions we need answered — this is the actual ask

We confirmed against the live sandbox that auth is an `api-key` header (not a bearer), and that card creation is `POST /issuing/users/{userId}/cards`. Three things we can't resolve ourselves:

1. Our collateral contract isn't linked. `GET /issuing/users/{userId}/contracts` returns `[]`, and `GET /contracts/{id}` on the ID from our credentials sheet returns 403. Does it need attaching to the user, or funding with RUSD first? *Without this an issued card has no spending power — it's the one thing between us and money genuinely moving.*

2. What status value deactivates a card? `PATCH /issuing/cards/{id}` exists and accepts a body, but `{"status":"inactive"}` returns 400. We retire the card after settlement (your own framing — "retired automatically once the job is done") and it works locally; we just need the enum.

3. What goes in `configuration` to set a spending limit and a short expiry? It comes back as `{"currency":"usd"}`. Worth flagging: creating a card with the *minimum* body gives an active card with **no spending limit and a six-year expiry**. We always want to send an explicit limit — we just don't know the shape.

The question we'd most like your honest answer to

We think we built one idea. We're worried it *reads* as many.

> **Does this look like one thing done properly, or like a lot of things bolted together? And if you had four minutes to show it to someone, what would you cut?**

We would genuinely rather hear "cut half of it" tonight than find out on stage.

If they want to see it

One page, one flow: run a task → it's checked → a card exists, or doesn't → try to break it yourself. The last one is a challenge panel that hands you the attack: change the supplier, skim the price, split a purchase in two to duck the approval limit. Eleven checks, and the "defeated" counter has never moved.